Classification on Imbalanced Datasets

Assoc. Prof. Kitsana Waiyamai, Ph. D.

Dept. Of Computer Engineering
Faculty of Engineering, Kasetsart University
Bangkok, Thailand

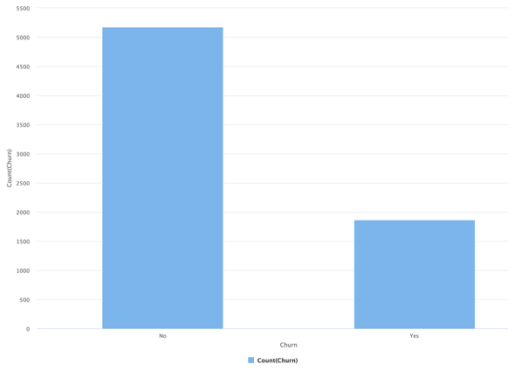


Imbalanced Datasets



- Many datasets are imbalanced.
- They contain **imbalanced** amount of data.
- This means one class, the **majority class**, has significantly **more samples** than another class, the **minority class**.
- Most of the time, class of interest is **Minority Class**.

- For example,
  - **Number of defective products** compared to number of non-defective products.
  - **Number of fraudulent transactions is much smaller** than number of normal transactions.
  - **Churn = Yes** VS Churn = No
  - **Minority Class** VS Majority Class
  - Etc.



Example of Imbalanced Churn Dataset

Name	Type	Missing	Statistics			Filter (21 / 21 attributes): Search for Attribu
DeviceProtection	Nominal	0	Least No inter [...] ce (1526)	Most No (3095)	Values No (3095), Yes (2422), ...[1 more]	
TechSupport	Nominal	0	Least No inter [...] ce (1526)	Most No (3473)	Values No (3473), Yes (2044), ...[1 more]	
StreamingTV	Nominal	0	Least No inter [...] ce (1526)	Most No (2810)	Values No (2810), Yes (2707), ...[1 more]	
StreamingMovies	Nominal	0	Least No inter [...] ce (1526)	Most No (2785)	Values No (2785), Yes (2732), ...[1 more]	
Contract	Nominal	0	Least One year (1473)	Most Month-to-month (38...	Values Month-to-month (3875), Two year (1695), ...[1 more]	
PaperlessBilling	Nominal	0	Least No (2872)	Most Yes (4171)	Values Yes (4171), No (2872)	
PaymentMethod	Nominal	0	Least Credit c [...] c) (1522)	Most Electronic check (2365)	Values Electronic check (2365), Mailed check (1612), ...[2 more]	
MonthlyCharges	Real	0	Min 18.250	Max 118.750	Average 64.762	
TotalCharges	Real	11	Min 18.800	Max 8684.800	Average 2283.300	
Churn	Nominal	0	 Open visualizations	Least Yes (1869)	Most No (5174)	Values No (5174), Yes (1869) Details...

Example of Imbalanced Dataset

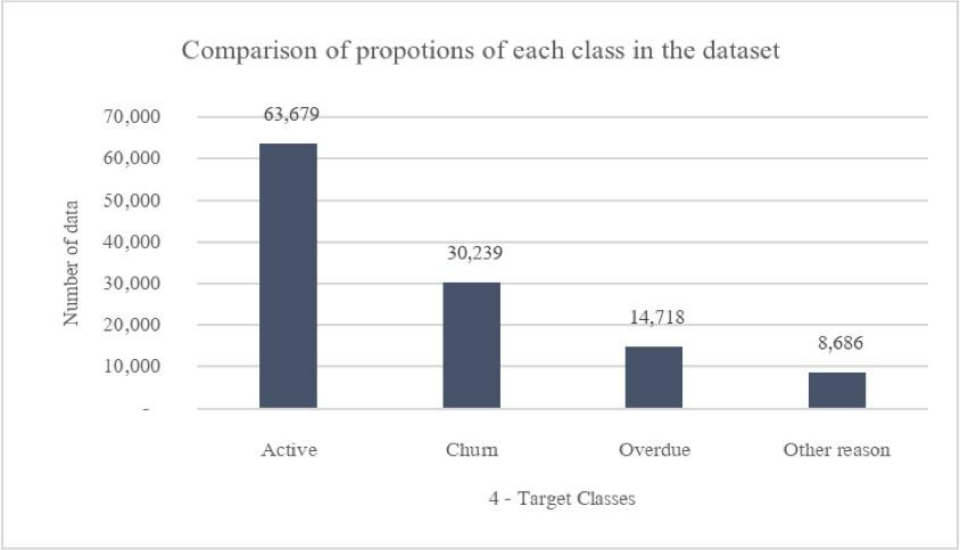
Department Name	Number of occurrences
Computer_Engineering	162
Civil_Engineering	400
Electrical_Engineering	799
Chemical_Engineering	349
Mechanical_Engineering	200
Environ_Engineering	375

Case Study: Internet Service Cancellation Prediction

Distribution of target classes:

- Active: 63,679 samples
- Claim: 30,239 samples
- Overdue: 14,718 samples
- Other reason: 8,686 samples

This is a typical example of **multi-class** data imbalance.



Understanding Data Imbalance

- **Data imbalance** refers to the situation where the distribution of samples across classes is significantly uneven.
- Common in real-world machine learning applications like fraud detection or rare disease diagnosis
- Models tend to favor the **majority class**, ignoring **minority class** patterns
- Leads to **low performance metrics such as recall and F1-score for minority classes**

Class Imbalance Ratio

- **Class Imbalance ratio**
  - Quantifies the **skewed distribution of classes**.
  - Quantifies the **extent of the class imbalance** problem.
  - Refers to the ratio of the number of examples in the majority class to the number of examples in the minority class.
- **For example**,
  - 10000 examples of class A (Majority Class)
  - 100 examples of class B (Minority Class)
  - **Imbalance ratio (IR)** is 10000/100 or approximately 100:1

- **Levels of class imbalance**
  - There's no strict threshold to define the severity of class imbalance.
  - However, some general classifications are used
    - **Moderate Imbalance:**
      - Ratio between 10:1 and 100:1
    - **Extreme Imbalance:**
      - Ratio above 1000:1

Measuring Imbalance: Imbalance Ratio (IR)

Imbalance Ratio (IR) is used to quantify the imbalance level between classes:

IR = (Number of majority class samples) / (Number of minority class samples)

Example from dataset:

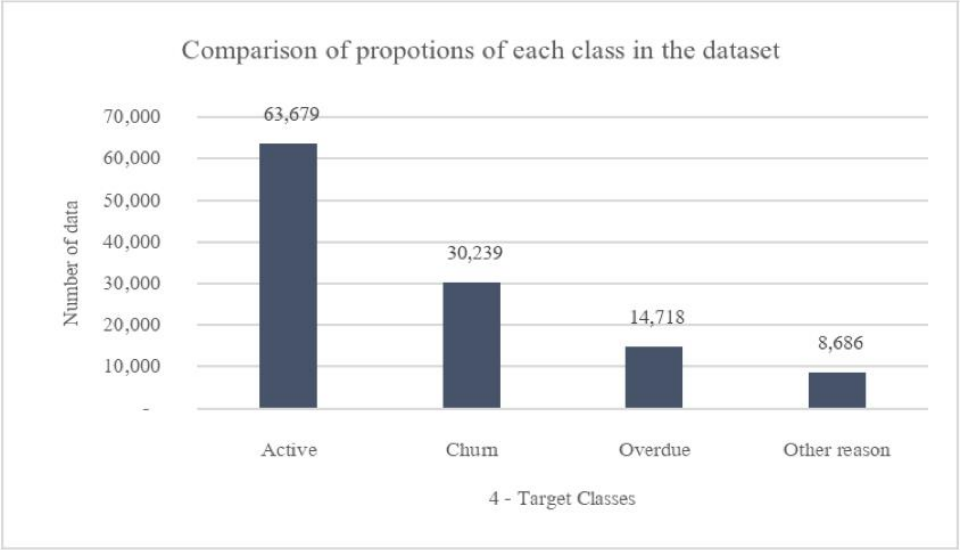
- Active vs. Claim: IR = 2.10
- Active vs. Overdue: IR = 4.33
- Active vs. Other reason: IR = 7.33

These values show varying degrees of imbalance across classes.

Common categorization of imbalance severity:

- **Mild Imbalance**: IR < 10:1
- **Moderate Imbalance**: IR between 10:1 and 100:1
- **Severe Imbalance**: IR > 100:1
- **Extreme Imbalance**: IR > 1000:1

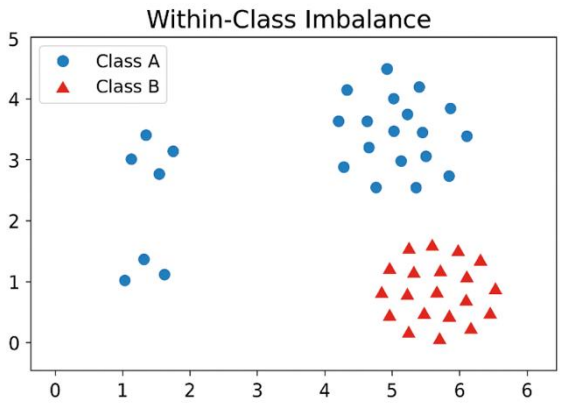
Even mild imbalance can significantly affect model performance.

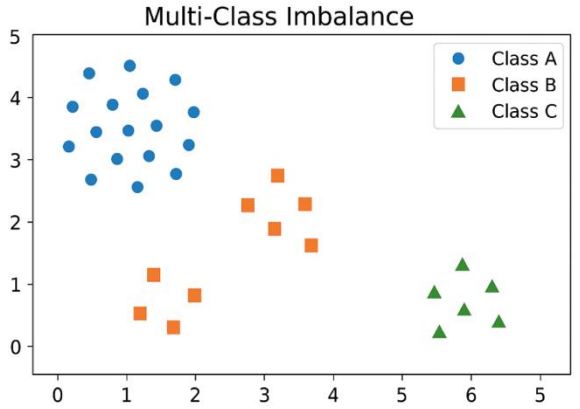


Types of Data Imbalance

Main types of imbalance:

- **Between-Class Imbalance**:
  - Unequal class sizes (most common, measured by IR)
- **Within-Class Imbalance**: Uneven distribution inside a class (e.g., small disjuncts, sparse areas)
- **Multi-Class Imbalance**: Involves multiple classes with complex hierarchies or relationships





Impact of Class Imbalance

- **Biased towards the Majority Class:**
  - Models trained on imbalanced data tend to perform well on the majority class but poorly on the minority class.
  - **Difficulties Learning Minority Class Characteristics:**
    - With fewer samples, the model might struggle to capture the defining features of the minority class.
  - A high imbalance ratio (IR), where the minority class has significantly fewer examples than the majority class, can pose challenges for analytics techniques because they tend to be **biased towards the majority class**.
  - This can result in poor classification performance, as the **model may simply predict the majority class most of the time**.

Impact of Class Imbalance

- Models generated from these data may not classify the smallest amount of data well due to
- Too few examples of the minority class

  => Very bad accuracy of this class.

  => Low Precision, Recall and F1-score even with High Accuracy

	Predicted: Legit (0)	Predicted: Fraud (1)
Actual: Legit (0)	950	30
Actual: Fraud (1)	15	5

Metric	Class 0 (Legit)	Class 1 (Fraud)
Precision	98.45%	14.29%
Recall	96.94%	25.00%
Accuracy	95.50% (overall)	

- Can use Cost-Matrix for model evaluation.

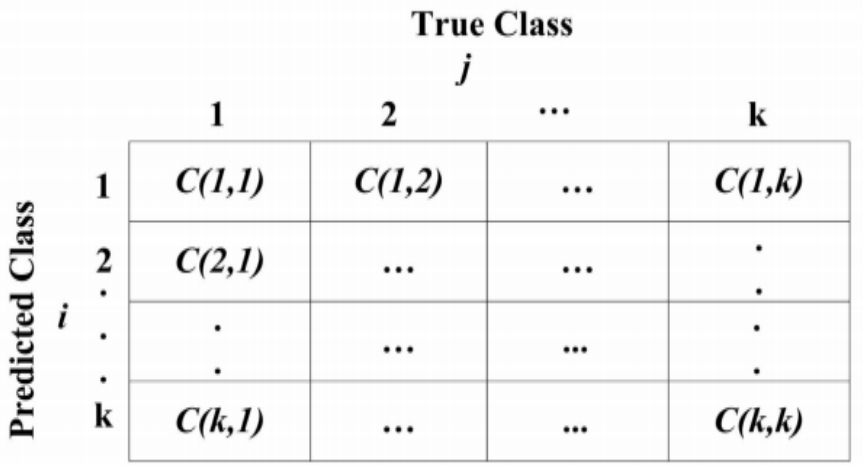
Techniques to Address Class Imbalance

- **Using Algorithms** Less Affected by Imbalance: Certain algorithms like **Ensemble** techniques: **XGBoost**, **RandomForest**, **Boosting** might be more robust to imbalanced data.
- **Cost-Sensitive Learning**: Assigning higher costs to misclassifying the minority class during model training.
- **Data over-sampling**: Modifying the dataset by over-sampling the minority class to achieve a more balanced distribution.
- **Data under-sampling**: Modifying the dataset by under-sampling the majority class to achieve a more balanced distribution.

Randomforest

- Uses **multiple (ensemble) classification models** to predict. Joint decision of multiple classification models generates higher accuracy than using a single classification model.
- K different decision trees
  - Pick a random subset S, of training samples
  - For each subset => grow a full tree
  - Given, a new data point X
    - Classify X using each of the trees
    - **For example**, use majority vote: class predicted most often
- Dis-Advantages:
  - Difficult Interpretability,
  - overfitting can easily occur, need to choose the number of trees

Cost-Sensitive Methods



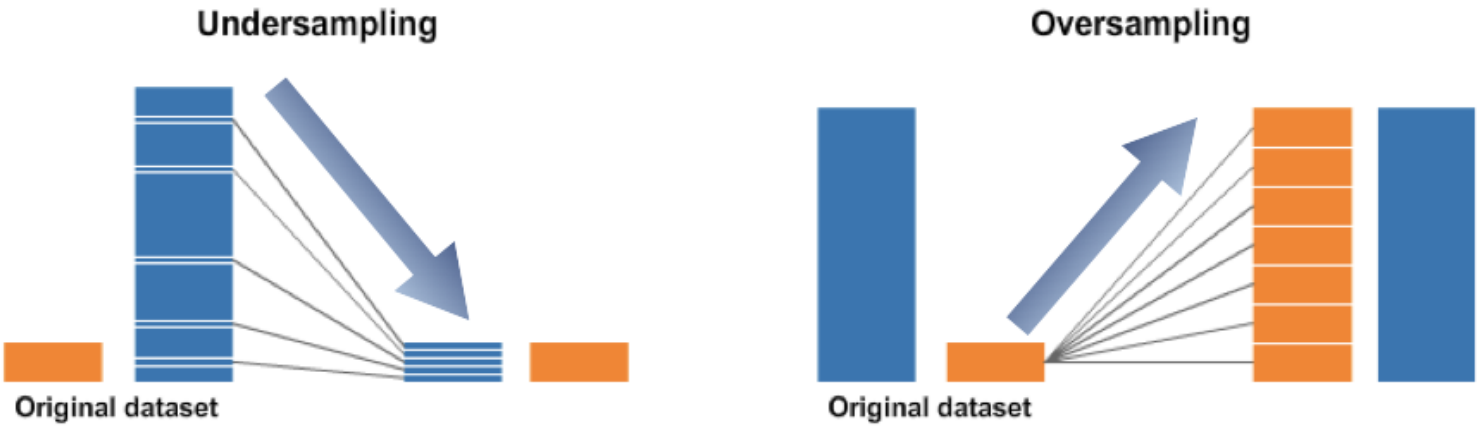
Predicted Class i \ True Class j	1	2	…	k
1	C(1,1)	C(1,2)	…	C(1,k)
2	C(2,1)	…	…	. .
. . .	. .	…	…	. .
k	C(k,1)	…	…	C(k,k)

Multi-class cost matrix

- No cost for correct classification
- Cost of misclassifying minority examples is higher than the contrary case
- Minimizes the overall cost on the training data set

Data Re-Sampling Methods

- **Sampling fixing size**
- **Under-Sampling Methods**
  - **Random Under-sampling (RUS)**: Partially select data from the majority class so that the number is close to the minority class. For example, RUS (Random Under-sampling Technique), Near-Miss, etc.
- **Over-Sampling Methods**
  - **Random Over-sampling (ROS)**: Increase the number of minority class.
  - **Replication**, this method does not really get new data.
  - **Synthetic generation** of minority data.
  - For example, SMOTE (Synthetic Minority Over-sampling Technique)
- **Hybrid Methods**
  - Do both Over-Sampling and Under-Sampling



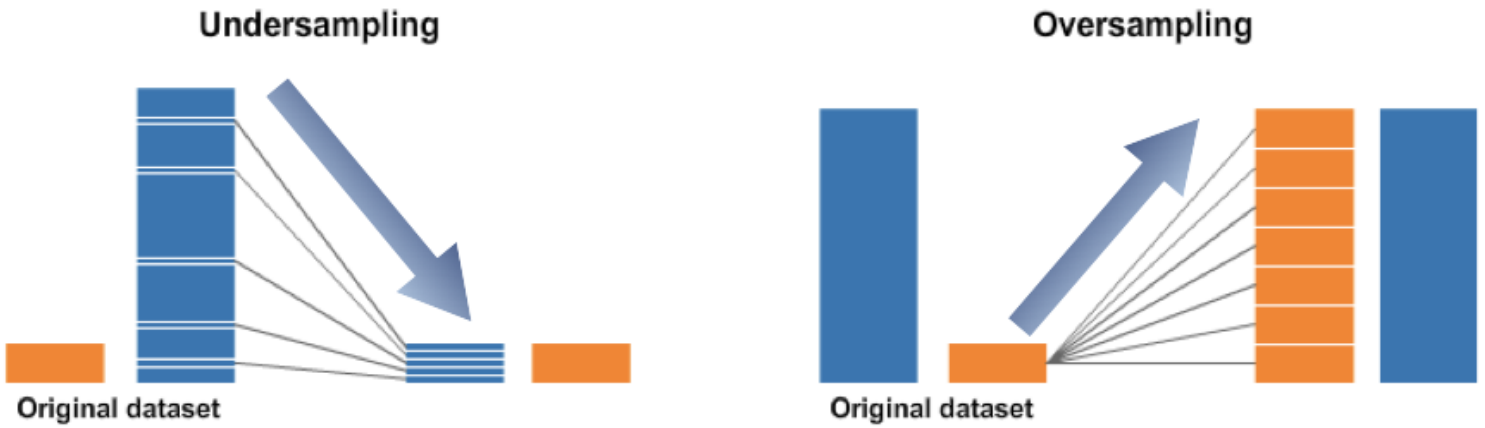
Over-sampling VS Under-sampling

- **Random under-sampling**: randomly remove a set E sampled from the majority class

  Cons: cause the classifier to miss important concepts pertaining to the majority class

- **Random oversampling**: randomly add a set E sampled from the minority class

  Cons: leading to over-fitting



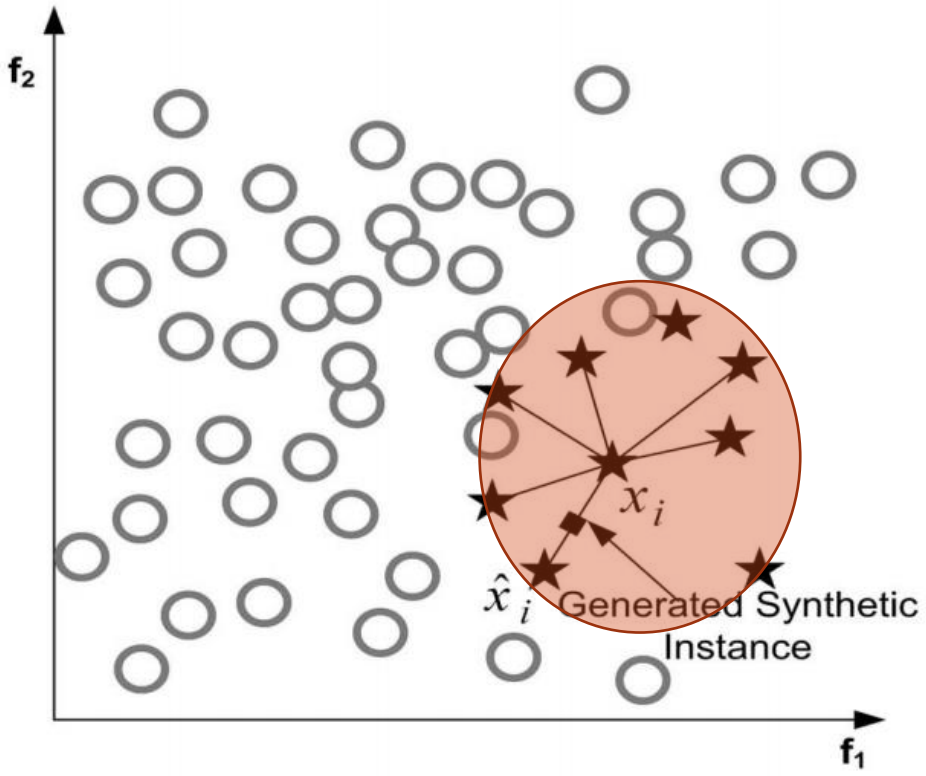
SMOTE: Synthetic over sampling with Data Generation

- A powerful method
- Create artificial data by

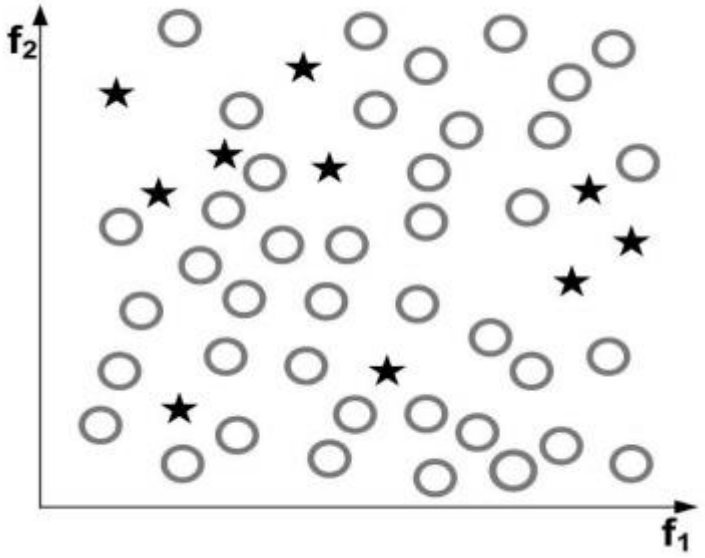
  Step1: For each minority sample X_i, find its K-nearest neighbors

  Step2: Randomly select one of the K-nearest neighbors, $\bar{X}_i$

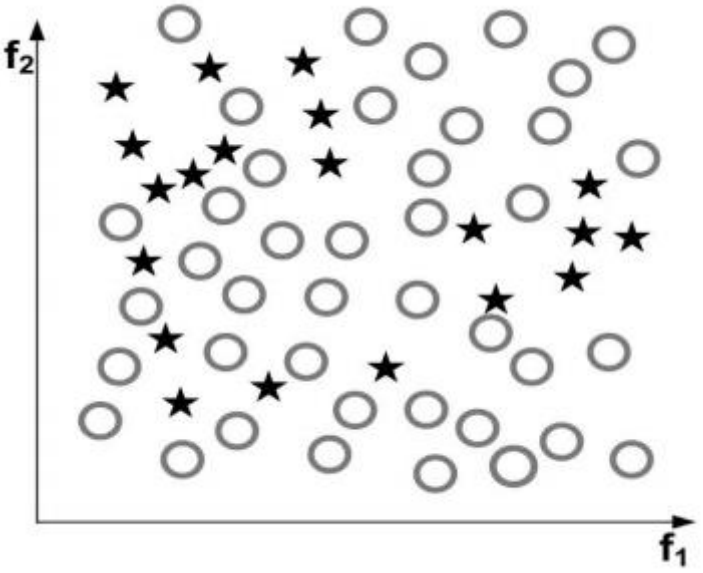
  Step3: Randomly generate X_{new} on the path form X_i to $\bar{X}_i$



SMOTE Over Sampling Technique

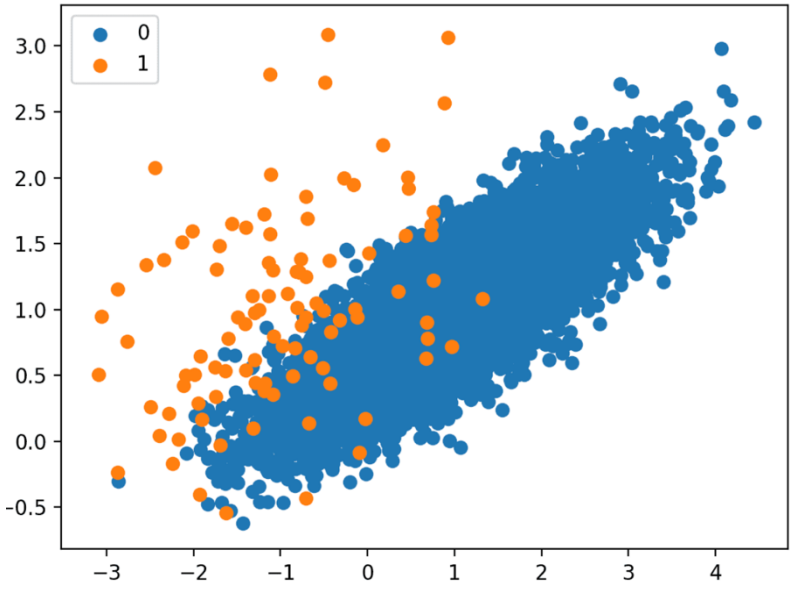


(a) Original data set distribution

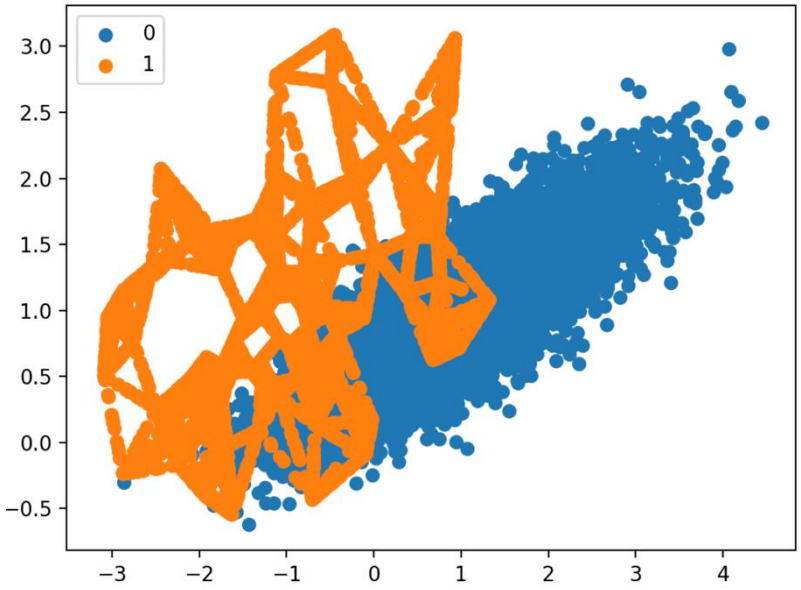


(b) Post-SMOTE data set

SMOTE Over Sampling Technique

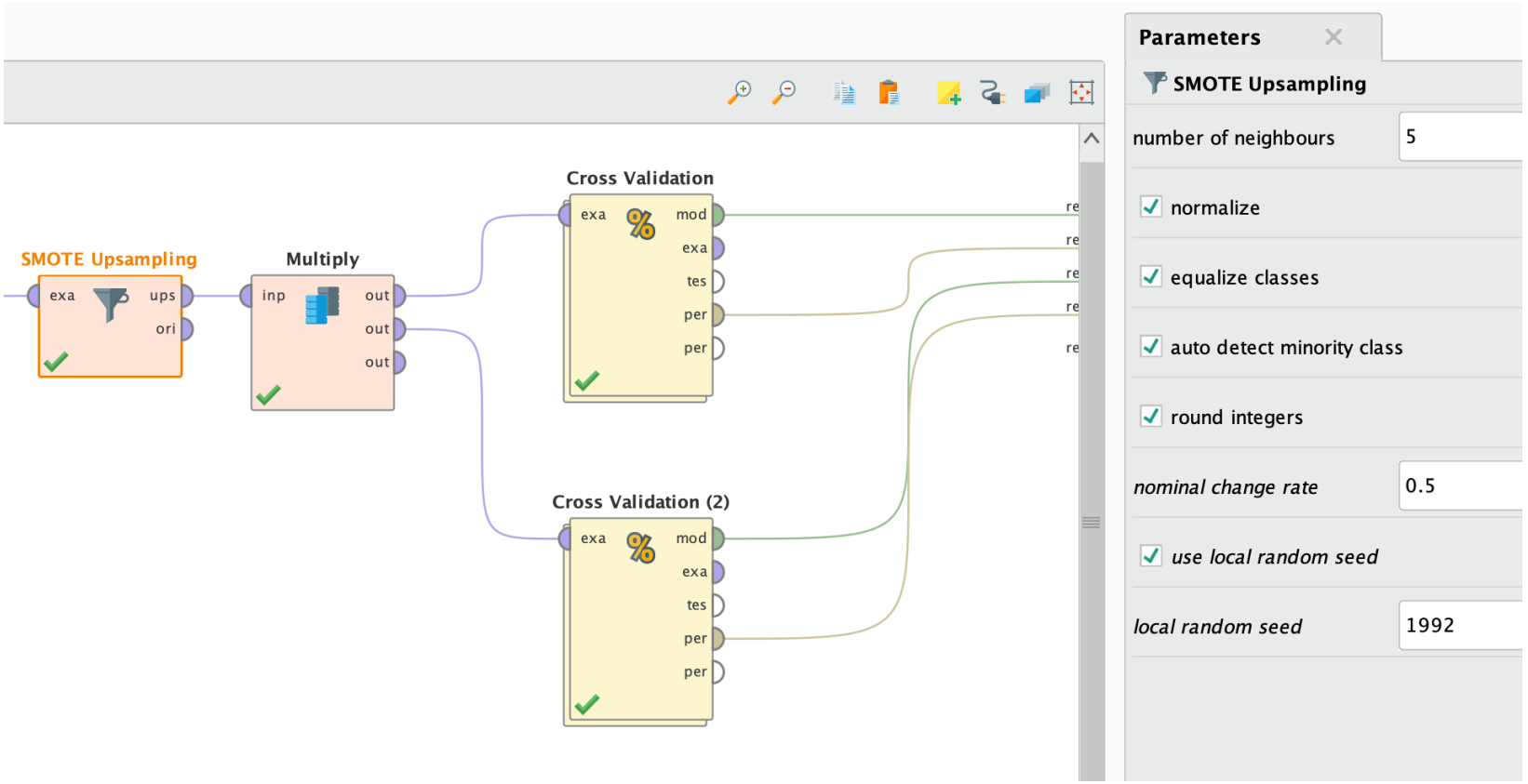


(a) Original data set distribution



(b) Post-SMOTE data set

SMOTE in Rapidminer



SMOTE in Python

```
from imblearn.over_sampling import SMOTE
X_resampled, y_resampled = SMOTE().fit_resample(X, y)

from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split

model = DecisionTreeClassifier()

X_train, X_test,  y_train, y_test = train_test_split(X_resampled, y_resampled)

model.fit(X_train, y_train)

model.score(X_test, y_test)
```

SMOTE in Python

SMOTE with 5-Fold Cross-Validation

Correct pattern: resample only the training fold

```
from sklearn.model_selection import StratifiedKFold, cross_validate
from sklearn.tree import DecisionTreeClassifier
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
model = Pipeline([
    ("smote", SMOTE(random_state=42)),
    ("classifier", DecisionTreeClassifier(random_state=42))
])

scores = cross_validate(
    model, X, y, cv=cv,
    scoring=["accuracy", "precision", "recall", "f1"]
)
```

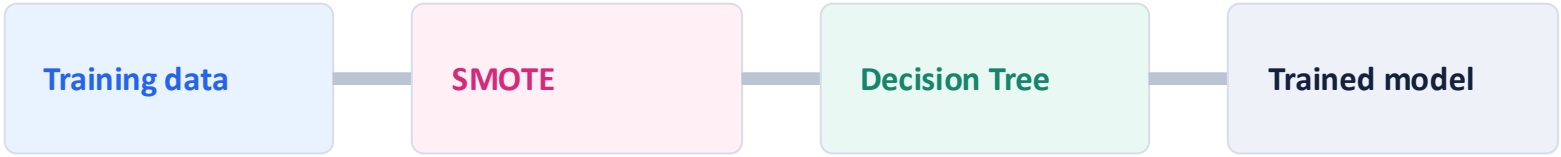
Do not apply SMOTE before Cross-Validation

SMOTE in Python

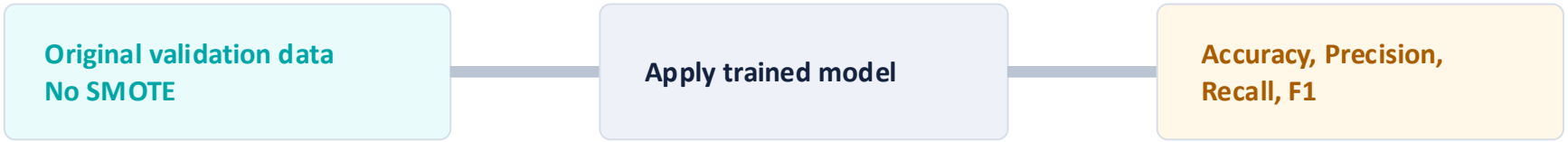
What happens inside each fold?

Repeat this process 5 times; each record becomes validation data once

TRAINING FOLD



VALIDATION FOLD

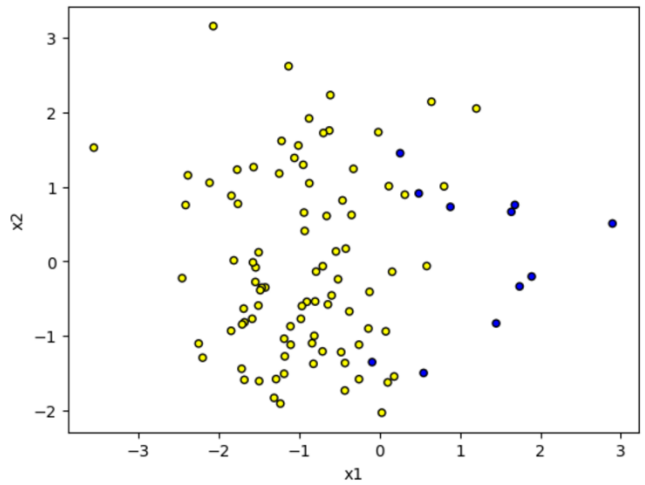


Key idea SMOTE learns synthetic samples from training data only. This keeps the evaluation fair.

Example: SMOTE in Python

```
1 X = df[['x1', 'x2']]
2 y = df['y']
3 X.plot.scatter('x1', 'x2', c=y, edgecolors='black')
```

```
<Axes: xlabel='x1', ylabel='x2'>
```




```
1 sm = SMOTE(k_neighbors=5)
2 X_new, y_new = sm.fit_resample(X, y)
3 X_new.plot.scatter('x1', 'x2', c=y_new, edgecolors='black')
```

```
<Axes: xlabel='x1', ylabel='x2'>
```

